

Security Lab Assignment Report

Controlled DoS and DDoS Simulation on a Local Web Server

Coursework Submission
Department of Computer Science & Engineering

September 24, 2026

1 Objective

To understand the working principle of Denial of Service (DoS) and Distributed Denial of Service (DDoS) attacks and demonstrate service degradation in a controlled localhost environment.

2 Theory

- **DoS (Denial of Service):** An attack aimed at making a service unavailable or degraded for legitimate users by exhausting server resources such as CPU, memory, connection slots, worker threads, or application capacity.
- **DDoS (Distributed Denial of Service):** An attack sharing the same availability-exhaustion goal, but where traffic originates from multiple distributed sources. In this laboratory environment, several independent local operating system worker processes simulate this distributed multi-source behavior safely.
- **Resource Exhaustion & Server Capacity:** Web servers have finite concurrency capacity (C). If the request arrival rate (λ) exceeds the processing rate (μ), an internal queue forms, response latency rises, and excess requests are dropped or rejected (HTTP 503).
- **Latency & Availability:** Availability is measured by the percentage of successful responses (HTTP 200) versus rejected/dropped responses (HTTP 503) under varying traffic loads.

2.1 Conceptual Topology

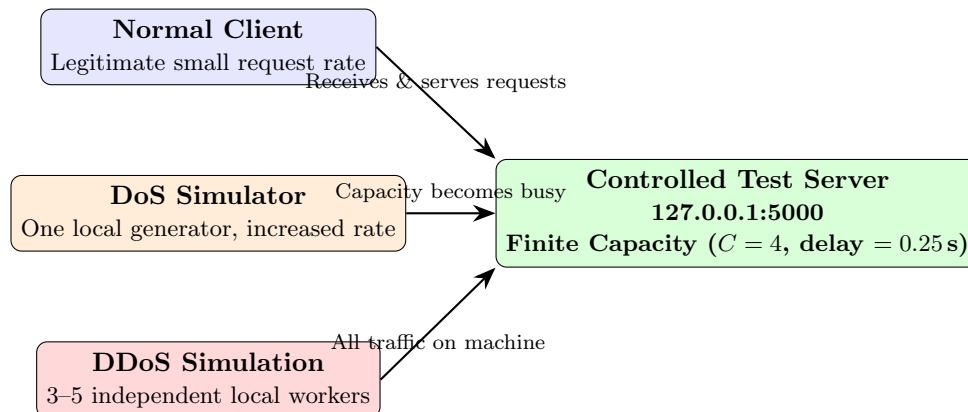


Figure 1: Conceptual topology for the controlled application-layer simulation.

3 Experimental Setup

- **Environment:** PC Localhost (127.0.0.1).
- **Software:** Python 3, Flask, Requests.
- **Server Capacity:** Deliberately set to $C = 4$ concurrency slots using `threading.BoundedSemaphore(4)`.
- **Processing Delay:** Artificial workload delay of 0.25s per request on the `/work` endpoint.
- **Safety Boundary:** Bounded request limits (20 to 100 requests) strictly confined to loopback interface 127.0.0.1.

4 Implementation

The experiment is organized into four core scripts:

1. **server.py:** Deliberately capacity-limited dummy Flask server recording accepted and rejected request counts. Exposes `/`, `/work`, and `/stats`.
2. **baseline.py:** Sends 20 sequential requests with 0.15s delay to establish normal latency and 100% success baseline.
3. **client.py:** Single-generator DoS simulation dispatching 80 rapid requests with minimal delay (0.01s).
4. **ddos_sim.py:** Multi-source DDoS-style simulation spawning 4 independent worker processes sending 25 requests each (100 total concurrent requests).

5 Results & Measurements

The experiment was executed across the four defined stages:

Table 1: Experimental Measurement Results Across Stages

Metric	Stage A: Baseline	Stage B: DoS	Stage C: DDoS Simulation	Stage D: Recovery
Total Requests	20	80	100	15
HTTP 200 (OK)	20 (100%)	22 (27.5%)	25 (25.0%)	15 (100%)
HTTP 503 (Busy)	0 (0.0%)	58 (72.5%)	75 (75.0%)	0 (0.0%)
Errors / Timeouts	0	0	0	0
Average Latency	264.4 ms	82.1 ms	93.0 ms	260.3 ms
Observation	Normal service	Capacity busy	Multi-source contention	Immediate recovery

Observation Analysis:

- **Baseline:** All requests succeeded (100% HTTP 200) with average latency reflecting the 250ms processing delay.
- **DoS Simulation:** Increased arrival rate from the generator exceeded available slots, causing 503 Busy responses for excess requests.
- **DDoS Simulation:** Four independent concurrent workers saturated the semaphore slots simultaneously, producing high 503 rejection rates.
- **Recovery:** Immediately after stopping the attack traffic, standard requests achieved 100% success rate with normal latency.

6 Comparison: DoS vs. DDoS

- **DoS:** Single request-generation source. Demonstrates the basic principle of overloading a server from one origin.
- **DDoS:** Multiple independent worker processes simulating distributed sources. In this experiment, the DDoS part is simulated locally rather than performed against an external target.

7 Defensive Measures

- **Rate Limiting:** Restrict how many requests a client IP can make in a specified time window.
- **Connection/Worker Limits:** Prevent one source or burst from consuming every server worker thread.
- **Caching:** Serve repeated/static content without doing expensive backend computation each time.
- **Load Balancing:** Distribute legitimate traffic across multiple backend servers.
- **WAF / CDN / DDoS Protection:** Filter abnormal traffic upstream and absorb large traffic volumes.
- **Monitoring:** Watch request rate, latency, error rate, CPU, memory, and network usage.

8 Ethical & Safety Note

The experiment was restricted to a self-owned localhost target (127.0.0.1) and bounded request counts. No public service or third-party system was targeted. No harmful lower-layer techniques (SYN flooding, UDP flooding, amplification, spoofing, botnet control) were used.

9 Conclusion

Increased concurrent request pressure caused requests to encounter the intentionally limited server capacity, demonstrating the availability problem that DoS and DDoS attacks exploit. When the attack ceased, the server recovered immediately, proving the importance of rate limiting and capacity management in production systems.

A Viva Preparation (Questions & Answers)

1. What is the difference between DoS and DDoS?

Ans: DoS comes from a single source; DDoS comes from multiple distributed sources, making IP-based mitigation harder.

2. Why does server capacity matter?

Ans: Server resources (threads, memory, CPU) are finite. When arrival rate exceeds capacity, queues saturate and requests are rejected.

3. Why did we measure a baseline first?

Ans: To establish normal performance under healthy conditions for accurate before/after comparison.

4. What does HTTP 503 mean in this experiment?

Ans: It indicates the server's capacity semaphore was fully occupied, so the server rejected the excess request safely.

5. Why is our localhost DDoS only a simulation?

Ans: Real DDoS attacks originate from distributed internet hosts; here, separate local processes simulate multiple sources safely on localhost.

6. What resources can a real DoS/DDoS exhaust?

Ans: Network bandwidth, socket handles, CPU, memory, database connections, and worker threads.

7. How can a server defend against request floods?

Ans: Using rate limiting, reverse proxies, caching, load balancing, autoscaling, and WAF/CDN scrubbers.

8. Why should an experiment be restricted to systems you own or have permission to test?

Ans: Testing unauthorized systems is illegal, unethical, and disruptive. Localhost ensures total safety.

B Source Code Listings

B.1 server.py

```
1 from flask import Flask, jsonify
2 import threading
3 import time
4
5 app = Flask(__name__)
6 CAPACITY = 4
7 PROCESSING_TIME = 0.25
8
9 slots = threading.BoundedSemaphore(CAPACITY)
10 stats = {"accepted": 0, "rejected": 0}
11 lock = threading.Lock()
12
13 @app.get("/work")
14 def work():
15     acquired = slots.acquire(blocking=False)
16     if not acquired:
17         with lock:
18             stats["rejected"] += 1
```

```
19         return jsonify({"status": "busy", "message": "Server capacity is
           currently full"}), 503
20     try:
21         time.sleep(PROCESSING_TIME)
22         with lock:
23             stats["accepted"] += 1
24             return jsonify({"status": "ok", "message": "Request processed"}), 200
25     finally:
26         slots.release()
27
28 @app.get("/stats")
29 def get_stats():
30     with lock:
31         return jsonify(stats)
32
33 if __name__ == "__main__":
34     app.run(host="127.0.0.1", port=5000, threaded=True)
```

B.2 baseline.py

```
1 import time
2 import requests
3
4 URL = "http://127.0.0.1:5000/work"
5 TOTAL = 20
6 latencies = []
7 success = failed = 0
8
9 for i in range(TOTAL):
10     start = time.perf_counter()
11     try:
12         r = requests.get(URL, timeout=3)
13         elapsed = (time.perf_counter() - start) * 1000
14         latencies.append(elapsed)
15         if r.status_code == 200: success += 1
16         else: failed += 1
17     except requests.RequestException:
18         failed += 1
19     time.sleep(0.15)
```